

Práctica Número 4

Variables

Objetivo: Conocer el manejo de identificadores en memoria y como lo definen e implementan los diferentes lenguajes.

Ejercicio 1:

a) Tome una de las variables de la línea 3 del siguiente código e indique y defina cuáles son sus atributos.

Elijo la variable “a”. Sus atributos son

- Nombre: a (identificador de la variable).
- Tipo: integer (define que tipo de valores almacena mi variable).
- Alcance: 4-16 (indica en que parte del programa es visible).
- L-valor: se asigna automáticamente (dirección de memoria asociada a la variable).
- R-valor: indefinido (contenido de la variable, antes de la línea 6 tiene basura).
- Tiempo de vida: 1-16 (es una variable local, salgo del Procedure y se termina su tiempo de vida).

```
</> pascal
Procedure Practica4();
var
  a, i: integer;
  p: puntero;
Begin
  a := 0;
  new(p);
  p := ^i;
  for i := 1 to 9 do
    a := a + i;
  ...
  p := ^a;
  ...
  dispose(p);
end;
```

b) Compare los atributos de la variable del punto a) con los atributos de la variable de la línea 4. ¿Qué dato contiene esta variable?

La línea 4 contiene una variable de tipo puntero, la diferencia con la variable de tipo integer es que la variable de tipo integer guarda un valor directo, mientras que la variable de tipo puntero guarda una dirección de memoria en donde se almacena otra variable. Además, si bien el L-valor de p es automático, el L-valor de p^ es dinámico.

Ejercicio 2:

a) Indique cuales son las diferentes formas de inicializar una variable en el momento de la declaración de la misma.

1. Inicialización por defecto: en algunos lenguajes, las variables reciben automáticamente un valor

inicial si no se especifica uno. Por ejemplo, los enteros suelen inicializarse en 0, los caracteres en blanco y otros tipos con valores predefinidos según el lenguaje. Esto evita errores por uso de variables sin inicializar, aunque puede ocultar problemas si el programador no es consciente de estos valores por defecto.

2. Inicialización en la declaración: consiste en asignar un valor a la variable en el mismo momento en que se declara. Por ejemplo, en C se puede escribir `int i = 0, j = 1;` mientras que en Ada sería `I, J : INTEGER := 0;` Esta práctica es recomendable porque hace explícito el estado inicial de las variables, mejora la legibilidad del código y reduce la probabilidad de errores.

3. Sin inicialización (ignorar el problema): en algunos casos, las variables no se inicializan y toman como valor inicial lo que haya en memoria en ese momento (una cadena de bits cualquiera). Esto puede generar comportamientos impredecibles y errores difíciles de detectar, por lo que requiere controles adicionales. En general, es una práctica desaconsejada en la mayoría de los contextos de programación.

b) Analice en los lenguajes: Java, C, Python y Ruby las diferentes formas de inicialización de variables que poseen. Realice un cuadro comparativo de esta característica.

Lenguaje	Por defecto	En declaración	Sin inicializar
Java	Sí (atributos de clase: 0, false, null).	Si (<code>int x = 5;</code>)	No permitido en variables locales.
C	Parcial (variables globales = 0).	Si (<code>int x = 5;</code>)	Si (variables locales tienen basura).
Python	No tiene.	Si (<code>x = 5</code>)	No permitido.
Ruby	Parcial (variables de instancia = nil).	Si (<code>x = 5</code>)	Variables locales no inicializadas: error!

Ejercicio 3: Explique los siguientes conceptos asociados al L-valor (dirección de memoria) de una:

a. Variable estática.

Una variable estática es aquella cuyo **L-valor se asigna una sola vez antes de la ejecución** del programa y **permanece fijo** durante toda su vida. Es decir, siempre ocupa el mismo lugar en memoria. Entran en esta categoría las variables globales. Ejemplo en C: `static int contador = 0;`

b. Variable automática o semiestática.

Son variables cuyo **L-valor se asigna en tiempo de ejecución**, generalmente al entrar a un bloque o función, y se libera al salir de él. Se llaman automáticas porque su creación y destrucción es gestionada automáticamente. Ejemplo en C: `void funcion() {int x = 10; // variable automática}`

c. Variable dinámica.

Son variables cuyo **L-valor se asigna y libera manualmente en tiempo de ejecución**, usando mecanismos como memoria dinámica (heap). **El programador controla su ciclo de vida**. Ejemplo.

En C: `int* p = (int*) malloc(sizeof(int));`
`*p = 5;`
`free(p);`

d. Variable semidinámica.

Son variables cuyo **L-valor se asigna en tiempo de ejecución**, pero su tamaño o estructura puede depender de valores conocidos recién en ese momento. Sin embargo, **su liberación suele ser automática o controlada por el entorno**. Ejemplo en C: `void funcion(int n) {int arreglo[n]; // tamaño definido en ejecución}`

De al menos un ejemplo de cada uno.

Investigue sobre qué tipos de variables respecto de su L-valor hay en los lenguajes C y Ada.

1. C tiene principalmente:

- Variables estáticas → globales o con static.
- Variables automáticas → locales a funciones.
- Variables dinámicas → con malloc, calloc, free.
- Semidinámicas → arreglos de tamaño variable (VLA, desde C99).

2. Ada es más estricto y seguro en este aspecto:

- Variables estáticas → declaradas normalmente en bloques o paquetes.
- Variables dinámicas → mediante access (similar a punteros).
- Semidinámicas → estructuras con tamaño definido en ejecución (por ejemplo, arreglos con límites variables).
- No permite el uso de variables sin inicializar de forma insegura (fuerte control de tipos).

Ejercicio 4:

a. ¿A qué se denomina variable local y a qué se denomina variable global?

Una **variable local** es aquella que se declara dentro de un bloque o función y solo puede utilizarse dentro de ese ámbito. Su tiempo de vida suele limitarse a la ejecución de ese bloque.

Una **variable global**, en cambio, se declara fuera de cualquier función o bloque y puede ser

accedida desde cualquier parte del programa (según las reglas del lenguaje). Son todas las referencias a variables creadas en el programa principal, y su tiempo de vida abarca toda la ejecución del programa.

- b.** ¿Una variable local puede ser estática respecto de su L-valor? En caso afirmativo dé un ejemplo.

Sí, puede. Una variable local puede tener un L-valor estático, es decir, una dirección de memoria fija durante toda la ejecución, aunque su alcance siga siendo local, para esto debe declararse como estática (*static*).

Ejemplo en C

```
void funcion() {  
    static int contador = 0;  
    contador++; }
```

En este caso, contador es local a la función, pero su dirección en memoria no cambia y conserva su valor entre llamadas.

- c.** Una variable global ¿siempre es estática? Justifique la respuesta.

Una variable global puede no ser estática, puesto que el alcance de las mismas no condiciona el momento en que se alocan en memoria. Por ejemplo, en C podemos tener variables globales no estáticas mediante el uso de la palabra clave *extern*.

- d.** Indique qué diferencia hay entre una variable estática respecto de su l-valor y una constante.

Una variable estática respecto de su L-valor es aquella cuya dirección de memoria permanece fija durante toda la ejecución del programa, pero su valor puede cambiar.

Una constante, en cambio, es un dato cuyo valor no puede modificarse una vez definido. Las constantes no tienen un L-valor definido de manera tradicional. El L-valor se refiere a la dirección de memoria de una variable, pero las constantes no ocupan una posición de memoria de igual manera que las variables.

Ejercicio 5:

- a.** En Ada hay dos tipos de constantes, las numéricas y las comunes. Indique a que se debe dicha clasificación.

En Ada existen constantes numéricas y constantes comunes, y esta clasificación se debe al momento en que se realiza la ligadura de su valor (binding).

Constantes numéricas: su valor se determina completamente en tiempo de compilación. Se utilizan principalmente para definir valores conocidos de antemano y permiten mayor optimización. Los valores de las constantes numéricas se conocen antes de que el programa se ejecute.

Constantes comunes: representan valores que no son numéricos. Se definen utilizando una notación específica para cada tipo de constante, su valor puede depender de expresiones más generales. No se evalúan durante la compilación, sino que, en tiempo de ejecución, esto significa que los valores de las constantes comunes se conocen sólo después de que el programa se haya iniciado.

- b. En base a lo respondido en el punto a), determine el momento de ligadura de las constantes del siguiente código:

H: constant Float:= 3,5; → constante numérica con valor literal. Ligadura en tiempo de compilación.

I: constant:= 2; → constante numérica con valor literal, sin tipo especificado así que se considera integer. Ligadura en tiempo de compilación.

*K: constant float:= H*I;* → aunque depende de otras constantes, estas ya están determinadas en compilación, por lo tanto, su ligadura también ocurre en tiempo de compilación.

Ejercicio 6: Sea el siguiente archivo con funciones de C. Analice si llegaría a tener el mismo comportamiento en cuanto a alocação de memoria, sacar la declaración **(1)** y colocar dentro de **func1()** la declaración **static int x =1;**

```
Archivo.c
{
    int x=1; (1)

    int func1(){
        int i;
        for (i:=0; i < 4; i++) x=x+1;
    }

    int func2(){
        int i, j;
        /*sentencias que contienen declaraciones y sentencias que no contienen declaraciones*/
        .....
        for (i:=0; i < 3; i++) j=func1 + 1;
    }
}
```

En el código original, x es global, se puede acceder a x desde todo el programa, se asigna en el segmento de datos y su tiempo de vida es estático (dura todo el programa). L-valor estático (dirección fija de memoria).

Si coloco la declaración (1) dentro de la func1() x seguirá teniendo L-valor estático, pero ahora es local en func1().

La diferencia está en el alcance: en un principio x es global, se puede acceder desde cualquier parte del programa. Luego x es local de func1(), fuera de la función ya no es accesible, pero sigue siendo estática.

Ejercicio 7: Sea el siguiente segmento de código escrito en Java, indique para los identificadores si son globales o locales.

<pre>Clase 3.0Persona { public long id public string nombreApellido public Domicilio domicilio private string dni; public string fechaNac; public static int cantTotalPersonas; //Se tienen los getter y setter de cada una de las variables //Este método calcula la edad de la persona a partir de la fecha de nacimiento }</pre>	<pre>public int getEdad(){ public int edad=0; public string fN = this.getFechaNac(); return edad; } Clase Domicilio { public long id; public static int nro public string calle public Localidad loc; //Se tienen los getter y setter de cada una de las variables }</pre>
---	---

Todas las variables de la clase 3.0Persona son globales respecto a la clase (aunque algunas sean privadas o estáticas, pueden accederse desde cualquier línea dentro de la clase).

Las variables edad y fN son locales dentro del método getEdad()

Todas las variables de la clase Domicilio son globales dentro de esa clase.

Ejercicio 8: Sea el siguiente ejercicio escrito en Pascal

- a. Indique el rango de instrucciones que representa el tiempo de vida de las variables *i*, *h* y *mipuntero*.

Son variables globales (duran todo el programa) 1-16.

Ahora *mipuntero*[^] vive desde que lo inicializo con *new(mipuntero)* hasta el *dispose(mipuntero)* 10-13

- b. Indique el rango de instrucciones que representa el alcance de las variables *i*, *h* y *mipuntero*.

Pueden usarse en cualquier instrucción a partir de que las declaro (*i* a partir de la línea 6, *h* a partir de la línea 7...).

- c. Indique si el programa anterior presenta un error al intentar escribir el valor de *h*. Justifique

No hay error, ya que le asigno la suma de la variable *a* la que apunta el puntero *+ i* antes de hacer el *dispose*.

- d. Indique si el programa anterior presenta un error al intentar asignar a *i* la resta de *h* con *mipuntero*. Justifique.

Si da error porque ya hice *dispose(mipuntero)* entonces quiero restarle a un entero un valor *nil*.

- e. Determine si existe otra entidad que necesite ligar los atributos de alcance y tiempo de vida para justificar las respuestas anteriores. En ese caso indique cuál es la entidad y especifique su tiempo de vida y alcance.
- f. Especifique el tipo de variable de acuerdo a la ligadura con el l-valor de las variables que encontró en el ejercicio.

Variable *mipuntero* → estática (pero que referencia a memoria dinámica), integer.

Variable *i* → estática, integer.

Variable *h* → estática, integer.

Variable *mipuntero*[^] → dinámica, integer.

```
Program Uno;
type tpuntero=^integer;
var
  mipuntero:tpuntero;
  i:integer;
  h:integer;
Begin
  i:=3;
  mipuntero:=nil;
  new(mipuntero);
  mipuntero^:=i;
  h:=mipuntero^+i;
  dispose(mipuntero);
  write(h);
  i:=h-mipuntero;
End.
```

Ejercicio 9: Elija un lenguaje y escriba un ejemplo:

- a. En el cual el tiempo de vida de un identificador sea mayor que su alcance.
- b. En el cual el tiempo de vida de un identificador sea menor que su alcance.
- c. En el cual el tiempo de vida de un identificador sea igual que su alcance.

Ejercicio 10: Si tengo la siguiente declaración al comienzo de un procedimiento:

int c; en C

var c:integer; en Pascal

c: integer; en ADA

Y ese procedimiento NO contiene definiciones de procedimientos internos. ¿Puedo asegurar que el alcance y el tiempo de vida de la variable “c” es siempre todo el procedimiento en donde se encuentra definida? Analícelo y justifique la respuesta, para todos los casos.

En los tres lenguajes (C, Pascal y Ada), si una variable c se declara al inicio de un procedimiento y no existen subprogramas internos, su alcance abarca todo el cuerpo del procedimiento y su tiempo de vida coincide con la ejecución del mismo (desde que se entra hasta que se sale), por lo que en estos casos ambos conceptos coinciden. Sin embargo, esto no puede afirmarse como regla general, ya que en ciertos lenguajes como C pueden declararse variables estáticas cuyo tiempo de vida se extiende a todo el programa, aunque su alcance siga siendo local al procedimiento.

Ejercicio 11: Responda Verdadero o Falso para cada opción. El tipo de dato de una variable es:

- Un string de caracteres que se usa para referenciar a la variable y operaciones que se pueden realizar sobre ella. → **Falso**: el tipo de dato no es un string de caracteres que referencia a la variable, eso sería el identificador de la variable.
- Conjunto de valores que puede tomar y un rango de instrucciones en el que se conoce el nombre. → **Falso**: eso sería el alcance, no el tipo.
- Conjunto de valores que puede tomar y lugar de memoria asociado con la variable. → **Falso**: esto sería el L-valor, no el tipo de variable.
- Conjunto de valores que puede tomar y conjunto de operaciones que se pueden realizar sobre esos valores. → **Verdadero**.

Escriba la definición correcta de tipo de dato de una variable: El tipo de dato de una variable es el conjunto de valores que puede tomar y el conjunto de operaciones que pueden aplicarse sobre dichos valores.

Ejercicio 12: Sea el siguiente programa en ADA, completar el cuadro siguiente indicando para cada variable de que tipo es en cuanto al momento de ligadura de su l-valor, su r-valor al momento de

alocación en memoria y para todos los identificadores cuál es su alcance y cual es su el tiempo de vida. Indicar para cada variable su r-valor al momento de aloación en memoria

```

1. with text_io; use text_io;
2. Procedure Main is
3.   type vector is array(integer range <>);
4.   a, n, p: integer;
5.   v1: vector(1..100);
6.   c1: constant integer := 10;

7.   Procedure Uno is
7.1     type puntero is access integer;
7.2     v2: vector(0..n);
7.3     c1, c2: character;
7.4     p, q: puntero;
7.5   begin
7.5.1    n := 4;
7.5.2    v2(n) := v2(1) + v1(5);
7.5.3    p := new puntero;
7.5.4    q := p;
7.5.5     ....
7.5.6    free p;
7.5.7     ....
7.5.8    free q;
7.5.9     ....
7.6   end;

8.   begin
9.     n := 5;
10.    ....
11.    Uno;
12.    a := n + 2;
13.    ....
14.   end

```

Ident.	Tipo	R-valor	Alcance	T.V
Main(2)	-	-	3-14	2-14
a (4)	Automática	Basura	5-14	2-14
n (4)	Automática	Basura	5-14	2-14
p (4)	Automática	Basura	5-7.4/8-14	2-14
v1 (5)	Automática	Basura	6-14	2-14
c1 (5)	Automática	Basura	6-7.3/8-14	2-14
Uno (7)	-	-	8-7.6	7-7.6
v2 (7.2)	Semidinámica	Basura	7.3-7.6	7-7.6
c1 (7.3)	Automática	Basura	7.4-7.6	7-7.6
c2 (7.3)	Automática	Basura	7.4-7.6	7-7.6
p (7.4)	Automática	Nil	7.5-7.6	7-7.6
q (7.4)	Automática	Nil	7.5-7.6	7-7.6
p^(7.5.3)	Dinámica	Basura	7.5-7.6	7.5.3-7.5.6
q^(7.5.4)	Dinámica	Basura	7.5-7.6	7.5.4-7.5.8

Aclaraciones:

Ident.= Identificador.

Tipo es el tipo de la variable respecto del l-value.

T.V. = Tiempo de Vida.

R-valor debe ser tomado al momento de la aloación en memoria.

El **alcance** de los identificadores debe indicarse

Ejercicio 13: Justifique la respuesta. El nombre de una variable puede condicionar:

- a) Su tiempo de vida. **Falso:** esto depende de como y donde está declarada, no del nombre.
- b) Su alcance. **Falso:** esto depende del bloque o contexto de declaración.
- c) Su r-valor. **Falso:** esto es el contenido o valor almacenado en la variable.
- d) Su tipo. **Falso:** esto lo define la declaración (integer, char, etc.)

Ejercicio 14: Sean los siguientes archivos en C, los cuales se compilan juntos. Indicar para cada variable de que tipo es en cuanto al momento de ligadura de su L-valor. Indicar para cada identificador cuál es su alcance y cuál es su el tiempo de vida. Indicar para cada variable su R-valor al momento de asignación en memoria.

ARCHIVO1.C

```
1. int v1;
2. int *a;
3. int fun2()
4. {
4.1 int v1, y;
5. for (y = 0; y < 8; y++)
6. {
6.1 extern int v2;
7. ...
8. }
9. }
10. main()
11. {
10.1 static int var3;
11.1 extern int v2;
12. int v1, y;
13. for (y = 0; y < 10; y++)
14. {
14.1 char var1 = 'C';
15. a = &v1;
16. }
```

ARCHIVO2.C

```
17. static int aux;
18. int v2;
19. static int fun2()
20. {
20.1 extern int v1;
21. aux = aux + 1;
22. ...
23. }
24. int fun3()
25. {
25.1 int aux;
26. aux = aux + 1;
27. ...
28. }
```

Ident.	Tipo	R-valor	Alcance	T.V
v1 (1)	Automática	0	2-4/10-12/18-28	1-28
a (2)	Automática	Null	3-17	1-28
a^	Dinámica	Basura	3-17	15-17
fun2 (3)	-	-	4-17	3-17
v1 (4.1)	Automática	Basura	5-9	3-9
y (4.1)	Automática	Basura	5-9	3-9
v2 (6.1)	Las extern sólo las analizo donde se definen			
main (10)	-	-	11-17	10-17
var3 (10.1)	Estática	0	11.1-17	<1-28>
v2 (11.1)	Las extern sólo las analizo donde se definen			
v1 (12)	Automática	Basura	13-17	10-17
y (12)	Automática	Basura	13-17	10-17
var1 (14.1)	Automática	Basura	15-16	13-16
aux (17)	Estática	0	18-25.1	<1-28>
v2 (18)	Automática	0	7/12-17/19-28	1-28
fun2 (19)	-	-	20-23	17-23
v1 (20.1)	Las extern sólo las analizo donde se definen			
fun3 (24)	-	-	25-28	17-28
aux (25.1)	Automática	Basura	26-28	24-28

- Tipo → las estáticas (static) todas automáticas, menos los punteros p^ (dinámicas).
- R-Valor → estáticas/globales inician con 0. Punteros p^ con null. Automáticas con basura.
- TV → Las estáticas duran todo el programa.

Ejercicio 15: Para javascript investigue la diferencia semántica para declarar una variable utilizando los modificadores `const`, `var`, `let` y la ausencia de cualquiera de estos. Compárelo con un lenguaje de su preferencia.

En JavaScript existen distintas formas de declarar variables que tienen diferencias semánticas importantes. La palabra clave `var` define variables con alcance de función, lo que significa que no respeta los bloques `{}` y puede ser accedida fuera de ellos dentro de la misma función; además, presenta hoisting, por lo que la variable es conocida antes de su declaración (inicializada como `undefined`), y permite la redeclaración. Por otro lado, `let` y `const` introducen variables con alcance de bloque, es decir, solo existen dentro del bloque donde se declaran; también tienen hoisting, pero no pueden utilizarse antes de su declaración debido a la llamada zona muerta temporal. La diferencia entre ambas es que `let` permite reasignar valores, mientras que `const` no permite cambiar la referencia una vez inicializada. La ausencia de cualquiera de estos modificadores genera una variable global implícita, lo cual es considerado una mala práctica porque puede producir errores difíciles de detectar.

Si se compara con un lenguaje como C, las diferencias son claras: en C todas las variables deben declararse explícitamente, no existe hoisting y el alcance está determinado estrictamente por el bloque en el que se define la variable. Además, no es posible crear variables globales implícitas, lo que hace que el lenguaje sea más estricto y predecible en este aspecto.